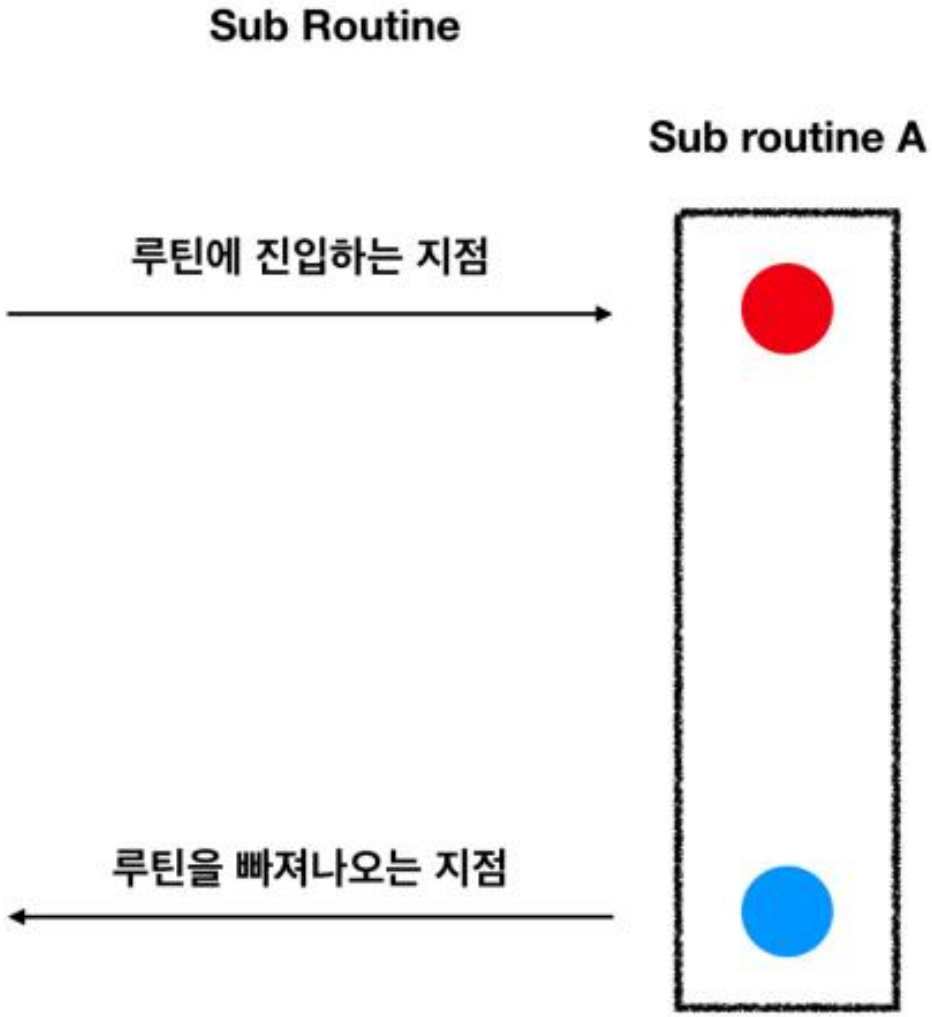
10 Coroutine

코루틴이란

1. 협력형 멀티캐스팅
   - Co + Routine : 협력하는 함수
   - 일반적인 함수 subroutine 실행
     - 맨 앞 진입
     - return 문이나 마지막
2. 동시성 프로그래밍
3. 쉬운 비동기처리



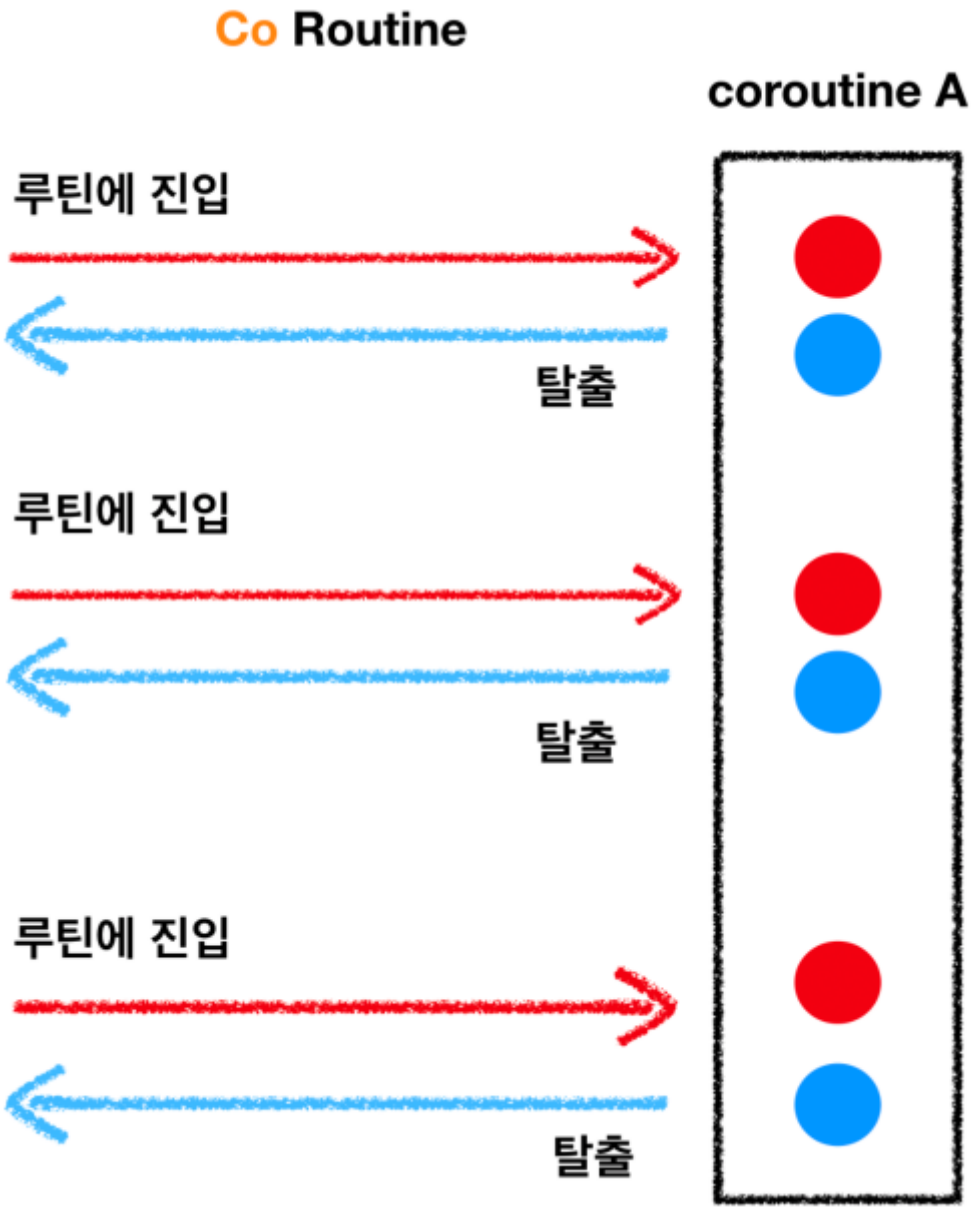
코루틴이란

1. 협력형 멀티캐스팅

- Co + Routine : 협력하는 함수
  - 코루틴 함수는 꼭 return 문이나 마지막 닫는 괄호를 만나지 않더라도 언제든지 중간에 나갈 수 있고, 언제든지 다시 나갔던 그 지점으로 들어올 수 있다.

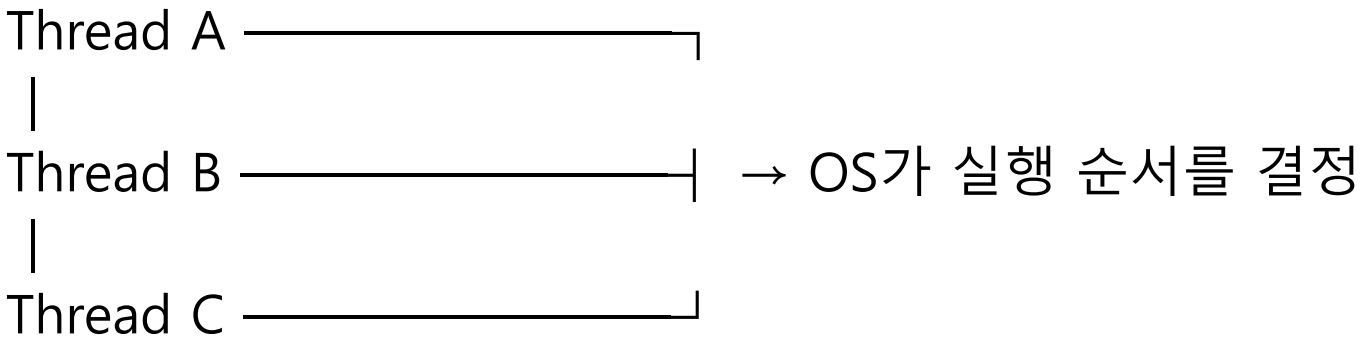
- 일반적인 함수
  호출 → 처음부터 실행 → 끝까지 실행 → 반환

- 코루틴 함수
  호출 → 처음부터 실행 → 일시 중지(탈출)
  → 다른 작업 수행 → 코루틴 재개 → 다음 실행
  → 일시 중지(탈출) → 다른 작업 수행
  → 코루틴 재개 → 반환



코루틴이란

- Thread와의 차이
  - 스레드:운영체제가 실행 관리



  - 스레드 A가 실행되다가 스레드 B로 전환되는 시점은 기본적으로 OS 스케줄러 결정
- 코루틴
  - 프로그램이 스스로 실행을 양보하는 방식

코루틴이란

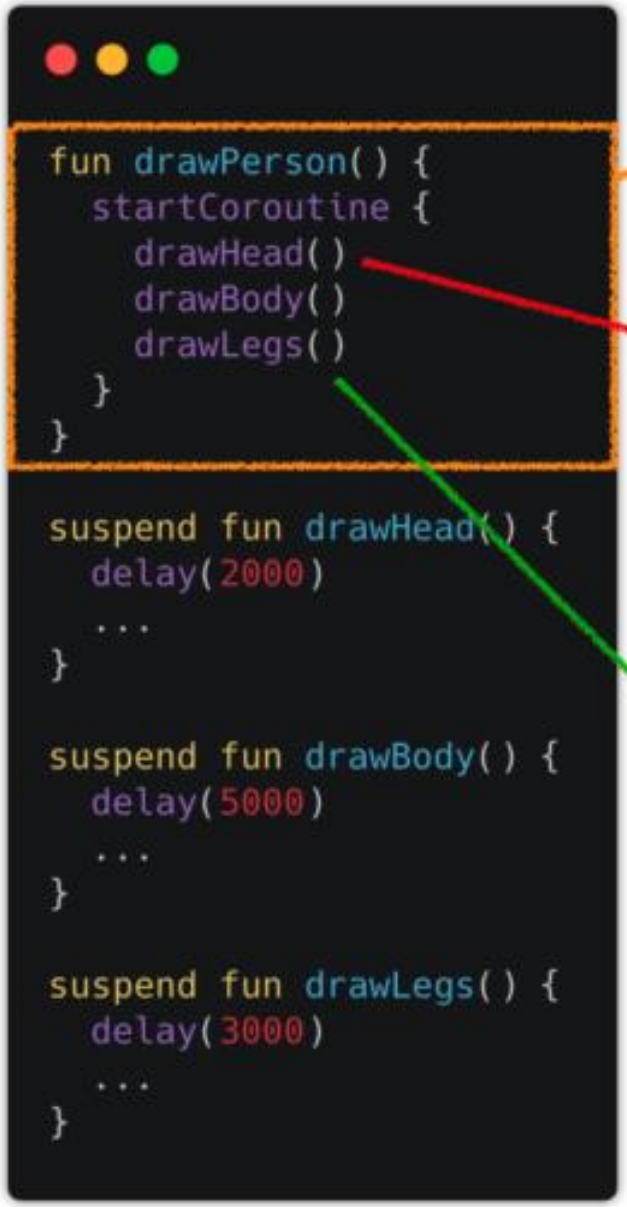
1. 협력형 멀티캐스팅

drawHead() 수행하면서
코루틴 탈출

→ 그 동안 다른 쓰레드 일 수행

⇩

2. 동시성 프로그래밍 가능해짐



1. 쓰레드의 main 함수가 drawPerson()을 호출하면 하나의 코루틴 블럭(함수)이 생성이 된다. drawPerson()은 언제든 진입, 탈출할 수 있는 자격이 주어진다.

2. 코루틴 함수가 실행되는 과정에서 suspend 키워드를 가진 함수를 만나게 되면, 더이상 아래 코드를 실행하지 않고 멈추고(suspend) 코루틴 block을 탈출한다.

3. 메인 쓰레드의 다른 코드들이 실행된다.
그러나 Head는 어디선가 계속 그려지고 있다.

4. 다른 코드들이 실행되다가도, drawHead가 끝이나면 다시 코루틴으로 진입해서 아까 멈춘 부분(drawHead) 아래부터 다시 실행된다.

코루틴이란

2. 동시성 프로그래밍이란(번갈아 실행)
    - 병렬 프로그래밍과 다름(CPU 2개)



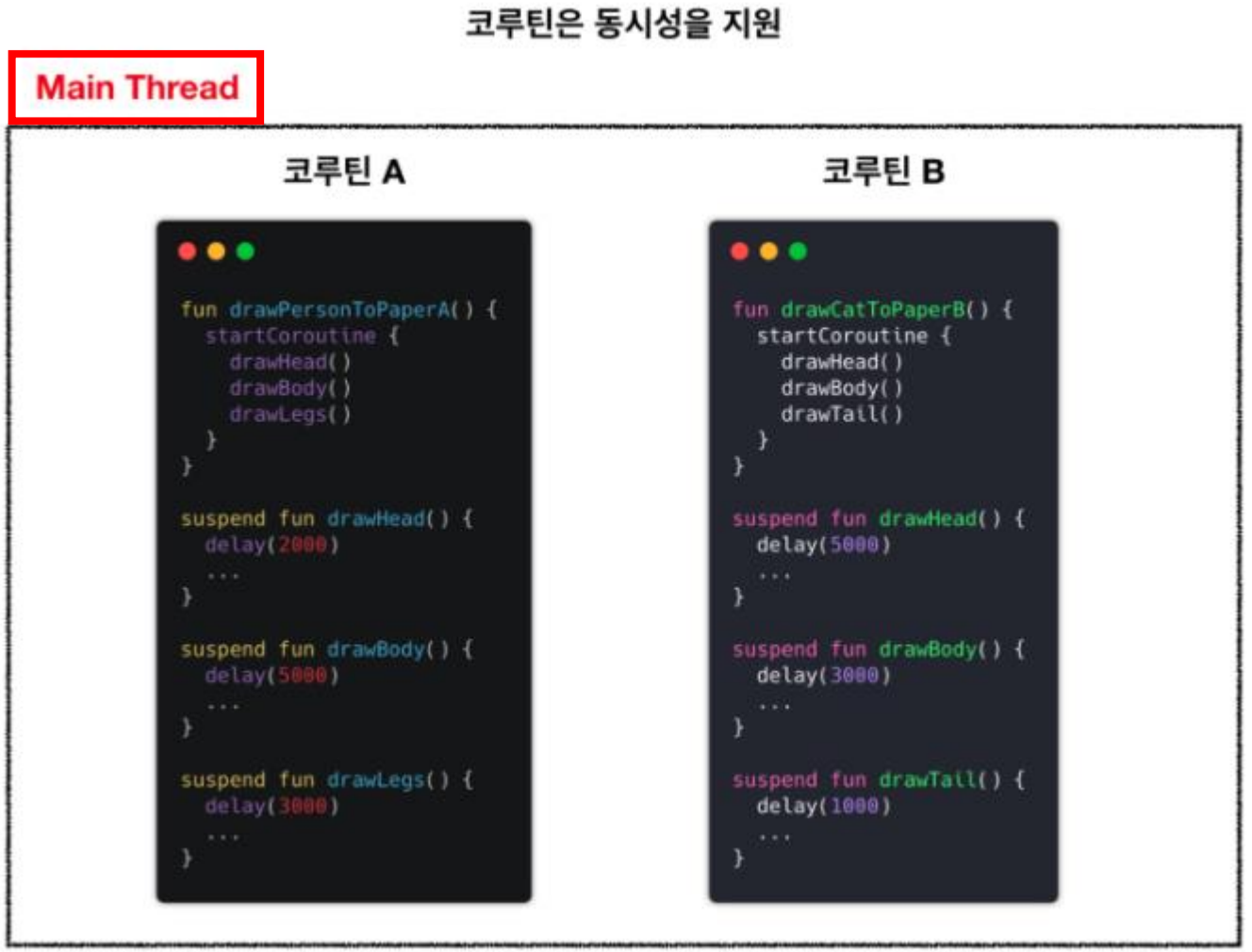
코루틴이란

2. 동시성 프로그래밍이란

- 하나의 쓰레드에 여러 코루틴 존재

- drawPersonToPaperA()가 호출되어 suspend함수인 drawHead()를 만나게 되면 이 코루틴을 잠시 빠져나간다.

- 메인쓰레드는 다른 suspend 함수들을 찾거나 resume되어지는 다른 코드들을 찾는다. delay(2000)는 쓰레드를 블락시키지 않았으므로 다른 일들을 할 수가 있다. 뿐만 아니라 drawHead() 함수 안에서 다른 쓰레드를 실행시킨다면 병행적으로도 실행이 가능하다. 왼쪽 코루틴을 빠져나온 쓰레드가 오른쪽 코루틴을 만나게 되어 또 한번 suspend 함수를 만나게 되면 아까 도화지 그림에서처럼 오른손에 펜을 쥐고 왼쪽과 오른쪽 도화지를 아주 빠르게 왔다갔다 하면서 그림을 그리는 것 같은 셈이다.



코루틴이란

2. 동시성 프로그래밍

- 멀티 쓰레드로 구현하면?

- 왼쪽에 조금, 오른쪽에 조금 그리기를 반복하기 위해선 1개의 CPU가 매번 쓰레드를 점유했다가 놓아주고, 새로운 쓰레드를 점유했다가 놓아주고를 반복해야 함
  → 컨텍스트 스위칭
  * 하나의 쓰레드에서 단순히 함수를 왔다 갔다 하는 코루틴과는 다르게 꽤 비용이 드는 작업임

단일 쓰레드로 멀티 쓰레드 효과 나타내는 코딩 방법!

쓰레드로 동시성을 지원했다면?

Thread A

```
fun drawPersonToPaperA() {
  startCoroutine {
    drawHead()
    drawBody()
    drawLegs()
  }
}

suspend fun drawHead() {
  delay(2000)
  ...
}

suspend fun drawBody() {
  delay(5000)
  ...
}

suspend fun drawLegs() {
  delay(3000)
  ...
}
```

Thread B

```
fun drawCatToPaperB() {
  startCoroutine {
    drawHead()
    drawBody()
    drawTail()
  }
}

suspend fun drawHead() {
  delay(5000)
  ...
}

suspend fun drawBody() {
  delay(3000)
  ...
}

suspend fun drawTail() {
  delay(1000)
  ...
}
```

Cpu core

코루틴이란

3. 쉬운 비동기처리

1. 8시 기상
2. 샤워하기
3. 옷입기
4. 출발하기
5. 회사도착하기
6. 일하기



```
suspend goCompany(person: Person) {
    val sleep = person
    try {
        val  sleepy = wakeUp(sleep)
        val  clean = takeShower(sleepy)
        val  dressed = putOnShirt(clean)
        val  onTheBus = getOnBus(dressed)
        val  atWork= finish(onTheBus)

        atWork.doWork()
    } catch (e: Exception){
        실패시 처리()
    }
}
```

회사 출근하기 전 루틴으로
각 단계는 시간이 얼마나 걸리는 지 모르며
순서대로 처리해야 함

- takeShower() 함수는 wakeUp() 함수가 끝나야지만 실행되고, putOnShirt() 함수는 takeShower() 함수가 끝나야만 실행된다.

→ goCompany라는 함수가 코루틴이기에 wakeUp을 만나면 wakeUp함수를 실행함과 동시에 잠시 goCompany를 빠져나간다. 그러다 wakeUp이 자신의 일을 끝마치면 다시 goCompany로 돌아올 수 있기 때문이다

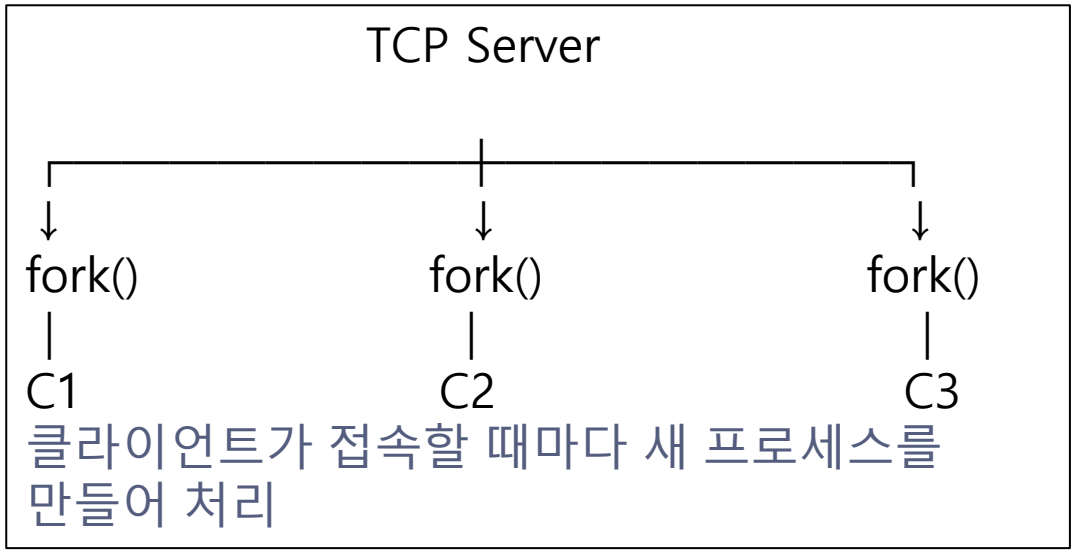
코루틴과 스레드의 차이점

- **스레드**:
  - 운영체제가 관리하며, 하나의 스레드가 실행 중에 다른 스레드의 실행을 강제로 중단시키는 선점적 방식을 사용합니다.
- **코루틴**:
  - 스레드 내부에서 동작하며, 실행을 중단하고 재개하는 협력적인 방식을 사용합니다.

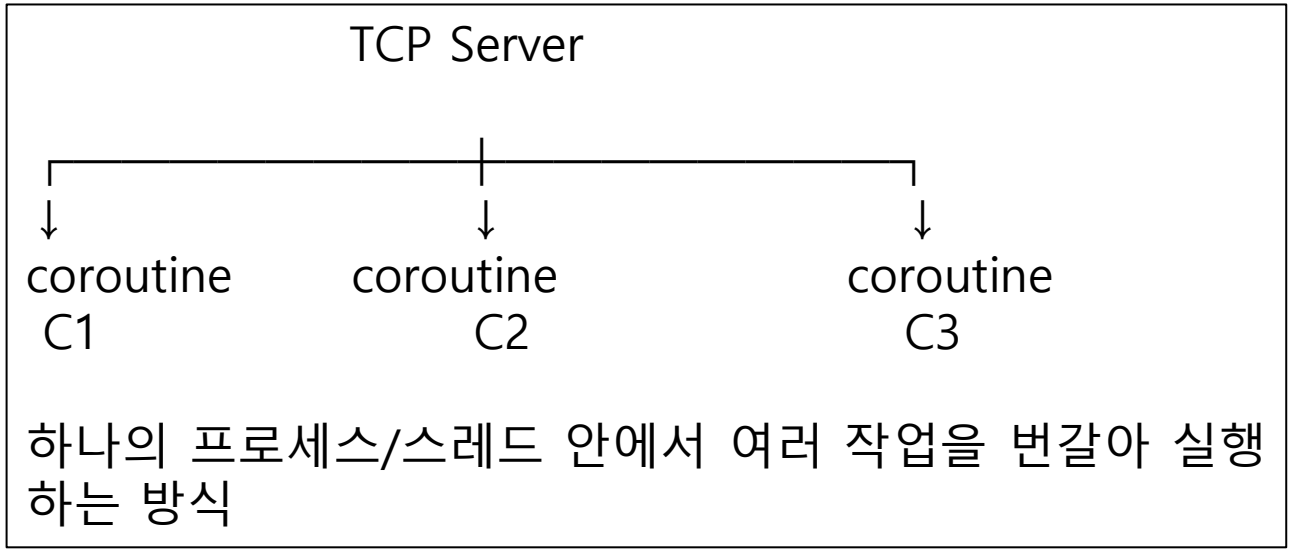


Thread와 Coroutine

여러 클라이언트 동시 연결 가능 소켓 프로그램



클라이언트가 접속할 때마다 새 프로세스를 만들어 처리



하나의 프로세스/스레드 안에서 여러 작업을 번갈아 실행하는 방식

CPU
- Coroutine A 실행
- Coroutine B 실행
- Coroutine C 실행
- Coroutine A 실행

- 네트워크 서버 구조 설계
  Event Loop
  + Coroutine
  + Async I/O

I/O 통신 종류

- 동기식 I/O

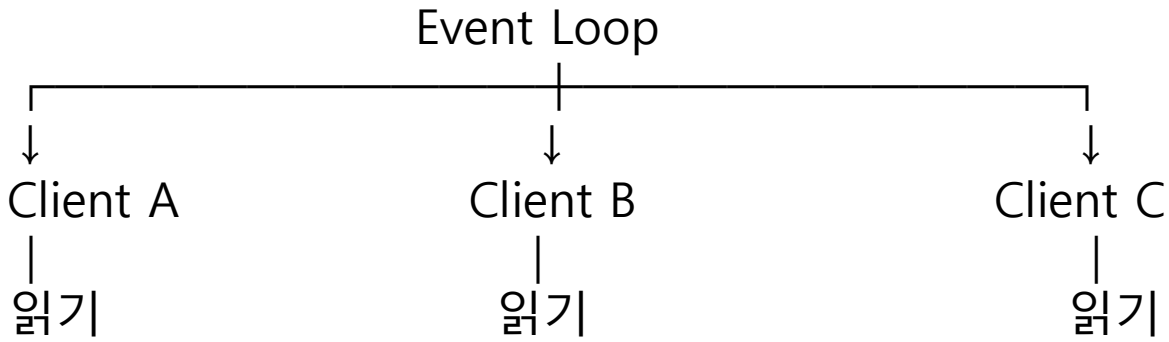
Server
 |
 | recv()
 ↓
 ┌──────────┐
 | 데이터 기다림 | ← 여기서 멈춤
 └──────────┘
 |
 | 데이터 도착
 ↓
recv() 완료되어 다음 코드 실행

- 비동기식 I/O

Server
 |
 | recv 요청 (제어가 바로 이동 가능)
 ↓
I/O 작업 시작
 |
 |──────────────────────────→ 데이터 기다리는 중
 |
 ↓
다른 작업 수행
 |
 ↓
또 다른 작업 수행
 |
 ↓
데이터 도착
 |
 ↓
처리

I/O 통신 종류

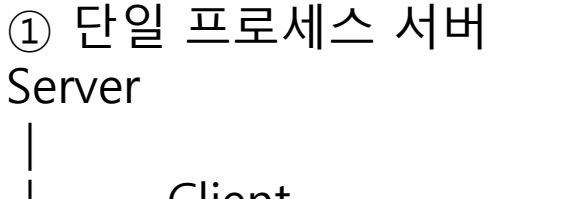
- Async I/O + Event Loop + Coroutine



Event Loop가 계속 돌아다니면서:
A에게 데이터가 왔나?
↓
B에게 데이터가 왔나?
↓
C에게 데이터가 왔나?
↓
A에게 데이터가 왔나?
↓
... 를 확인하고 이벤트가 발생한 작업 처리

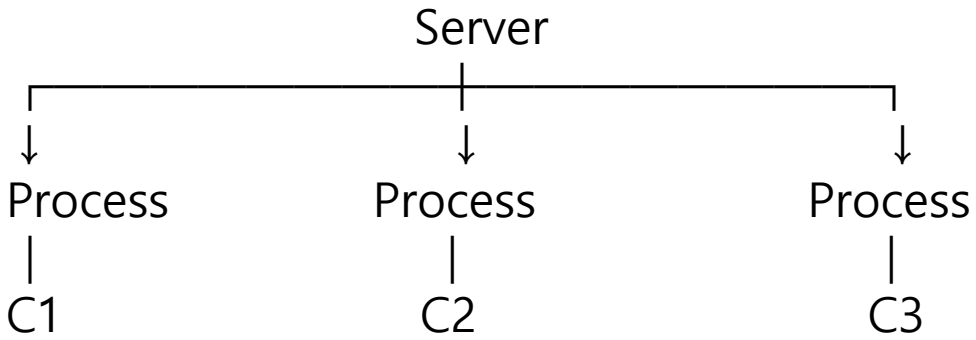
I/O 서버 설계

① 단일 프로세스 서버



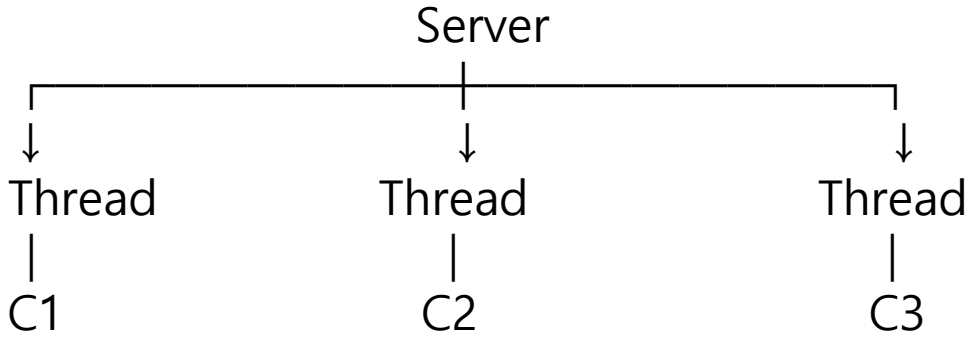
한 클라이언트만 처리 가능

② fork() 서버



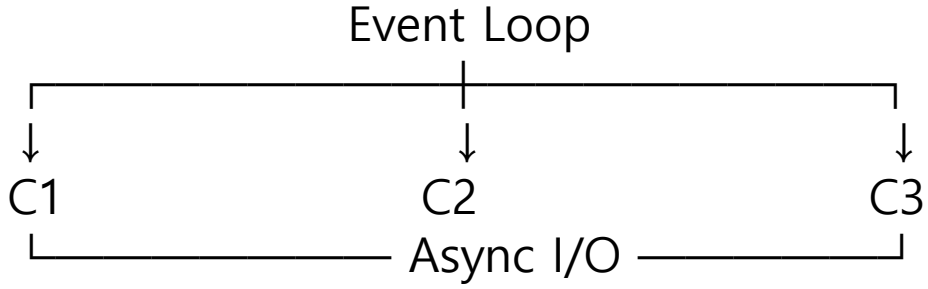
클라이언트마다 프로세스 제공

③ Thread 서버

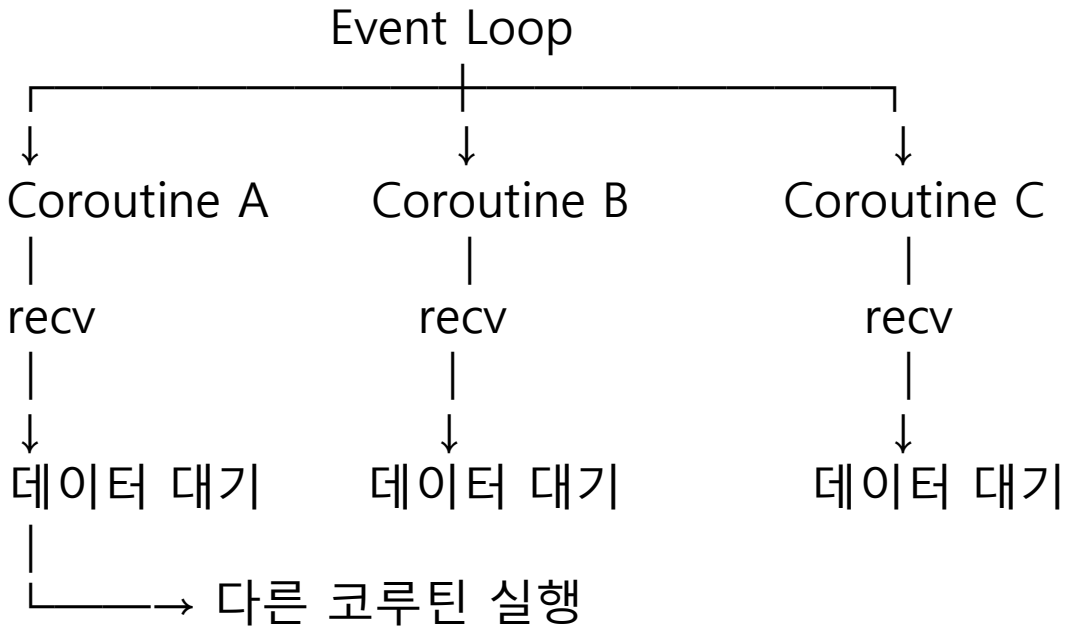


클라이언트마다 쓰레드 제공

④ Async I/O 서버



I/O 서버 설계



* Async I/O가 특히 강한 것은:
네트워크, 파일, 데이터베이스, 사용자 입력,
외부 API처럼 I/O를 기다리는 시간이 많은 작업임

- 앞으로 우리가 할 것
  ① 단일 TCP 서버 → ② fork 다중 클라이언트 → ③ 멀티스레드 서버 → ④ Blocking I/O의 문제 → ⑤ Non-blocking/Async I/O → ⑥ Event Loop → ⑦ Coroutine

코루틴을 지원하는 주요 프로그래밍 언어

언어	코루틴 지원 방식	주요 문법·기능
Python	표준 라이브러리 asyncio	async def, await
Kotlin	코루틴 라이브러리	suspend, launch, async
JavaScript	언어 내장 비동기 기능	async, await, Promise
C++	C++20 언어 기능	co_await, co_yield, co_return
C#	언어 내장 비동기 기능	async, await, Task
Lua	언어 내장 코루틴	coroutine.create(), yield(), resume()
Go	고루틴(Goroutine)	go, 채널
Rust	Future 기반 비동기 프로그래밍	async, .await, Future
PHP	버전 및 라이브러리에 따라 지원	Fiber(PHP 8.1 이상) 등
Ruby	Fiber 기반	Fiber, yield, resume

Python Coroutine

- async와 await 키워드를 사용해 코루틴 함수를 정의하고 실행하며, 이를 통해 여러 작업을 동시에 처리할 수 있습니다.

- **async**:
  - 함수 앞에 붙어 이 함수가 코루틴임을 명시합니다. 일반 함수와는 달리, 이 함수는 호출 시 즉시 실행되지 않고 코루틴 객체를 반환합니다.
- **await**:
  - 코루틴 내부에서 사용되며, 다른 코루틴의 실행이 끝날 때까지 기다립니다.
  - await가 호출되면 현재 코루틴은 실행을 일시 중지하고, 제어권을 이벤트 루프(event loop)에 넘깁니다
- **이벤트 루프(Event Loop)**:
  - 코루틴을 스케줄링하고 실행을 관리하는 역할을 합니다.
  - await를 만나 실행이 중단된 코루틴이 있으면, 이벤트 루프는 다른 준비된 코루틴을 실행합니다. 이 작업이 끝나면 다시 중단되었던 코루틴으로 돌아와 실행을 재개합니다.

Python Coroutine 예제

```
import asyncio
import time

# async 키워드로 코루틴 함수를 정의합니다.
async def say_after(delay, what):
    await asyncio.sleep(delay) # 딜레이가 끝날 때까지 일시 중지합니다.
    print(what)

async def main():
    print(f"시작: {time.strftime('%X')}")

    # 두 개의 코루틴 객체를 생성하고 동시에 실행을 예약합니다.
    task1 = asyncio.create_task(
        say_after(1, 'hello'))

    task2 = asyncio.create_task(
        say_after(2, 'world'))

    # 두 작업이 모두 끝날 때까지 기다립니다.
    await task1
    await task2

    print(f"종료: {time.strftime('%X')}")

# 메인 코루틴을 실행합니다.
asyncio.run(main())
```

실행 결과

```
시작: 12:30:00
hello
world
종료: 12:30:02
```

순차실행: hello 출력 1초
world 출력 2초 → 3초 실행
코루틴 실행: 두 작업이 동시 실행으로
2초 실행